



Information Systems Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Ambidexterity in Agile Distributed Development: An Empirical Investigation

Balasubramaniam Ramesh, Kannan Mohan, Lan Cao,

To cite this article:

Balasubramaniam Ramesh, Kannan Mohan, Lan Cao, (2012) Ambidexterity in Agile Distributed Development: An Empirical Investigation. Information Systems Research 23(2):323-339. <http://dx.doi.org/10.1287/isre.1110.0351>

Full terms and conditions of use: <http://pubsonline.informs.org/page/terms-and-conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2012, INFORMS

Please scroll down for article—it is on subsequent pages



INFORMS is the largest professional society in the world for professionals in the fields of operations research, management science, and analytics.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Ambidexterity in Agile Distributed Development: An Empirical Investigation

Balasubramaniam Ramesh

Computer Information Systems Department, Georgia State University,
Atlanta, Georgia 30302, bramesh@gsu.edu

Kannan Mohan

Department of Statistics and Computer Information Systems, Zicklin School of Business, Baruch College,
City University of New York, New York, New York, 10010, Kannan.mohan@baruch.cuny.edu

Lan Cao

Department of Information Technology and Decision Sciences, Old Dominion University,
Norfolk, Virginia 23529, lcao@odu.edu

Distributed software development has become a common reality with the advent of off-shore development and the need to be close to markets. Also, the dynamic nature of the environment in which businesses operate suggests the use of agile development methods. Whereas distributed software development requires the use of formal processes advocated by plan-driven approaches, rapidly changing environments are appropriate candidates for the use of agile development methods. This tension in agile distributed development poses conflicting demands between alignment and adaptability in the software development process. We conducted a multisite case study of three projects that use agile distributed development to examine how these organizations developed contextual ambidexterity—the ability to pursue conflicting demands simultaneously. Our findings, presented as a conceptual framework, indicate that conflicting demands between alignment and adaptability posed by agile distributed development can be addressed by a set of balanced practices that shape performance management and social context—two important antecedents of contextual ambidexterity.

Key words: agile development; ambidexterity; distributed development; qualitative case study

History: Brian Fitzgerald, Senior Editor; Jeffrey Parsons, Associate Editor. This paper was received August 4, 2009, and was with the authors for 5 1/2 months for 2 revisions. Published online in *Articles in Advance* July 26, 2011.

1. Introduction

Distributed development refers to information systems development in which project stakeholders are dispersed along any of the following boundaries: physical (or geographical), organizational, or temporal. Distributed development has become increasingly prevalent due to the rise in globalization (Komi-Sirvio et al. 2006) and the need to be close to markets. Although cost has been the primary driver of this trend, the dynamic nature of today's business environment with changing requirements and shortened time to delivery necessitate flexible development methods. Globally distributed software development typically relies on plan-driven approaches (Boehm and Turner 2003). Plan-driven methods are characterized by a systematic engineering approach that carefully adheres to specific processes in moving software through a series of representations from requirements to finished code (Boehm and Turner 2003). Agile methods are a set of software development methods that exhibit the ability to respond to change in a turbulent business environment (Highsmith and

Cockburn 2001). They employ short iterative cycles; actively involve users to establish, prioritize, and verify requirements; and rely on tacit knowledge within a team, as opposed to documentation. The use of agile methods in distributed software development requires reconciliation of practices drawn from both plan-driven and agile development methods, which are often considered to be diametrically opposite (Agerfalk and Fitzgerald 2006). Furthermore, the challenges posed by distributed development are often aggravated when agile practices are also implemented (Ramesh et al. 2006). These challenges range from the extent of documentation to the nature of communication (Simons 2006) to the extent of discipline needed in the development process (Parnas 2006). Whereas distributed development benefits from formalized processes recommended by plan-driven development approaches (Kurland and Egan 1999, Meadows 1996), agile development is more people oriented than process oriented. We investigate how incorporating both plan-driven and agile development methods to facilitate distribution and flexibility, respectively, poses

conflicting demands and how these can be addressed by balanced practices. The literature on organizational ambidexterity refers to the pursuit of such conflicting demands, which require trade-offs (Gupta et al. 2006). The conflicts posed by agile distributed development is consistent with achieving alignment and adaptability, a central focus in organizational ambidexterity (Gibson and Birkinshaw 2004). Thus, our study is guided by the following research question: How do software development teams develop ambidexterity within the context of agile distributed development?

Answering this question entails examining how software development teams mitigate the conflicting demands through the development of balanced practices. We accomplish this through a multisite case study and by drawing from the literature on ambidexterity. Our study extends the notion of ambidexterity in the context of agile distributed development by identifying specific practices that shape contextual ambidexterity. The paper is organized as follows: §2 discusses the conflicts posed by agile distributed development and presents the literature on organizational ambidexterity. Section 3 presents our research methodology. In §4 we present our results in a conceptual framework. In §§5 and 6, we discuss the results and our contributions to theory and practice.

2. Theoretical Background

Several challenges are introduced by distributed development in areas such as strategy, culture, communication, knowledge management, and project and process management (Herbsleb et al. 2005, Komi-Sirvio and Tihinen 2005, Mockus and Herbsleb 2001). Often, these challenges are addressed by enhancing formal controls that are used in a plan-driven approach. In addition, due to stringent time-to-market constraints and frequently changing requirements, organizations have increasingly recognized the need to incorporate flexibility in development through the use of agile methods. The use of both plan-driven and agile methods results in conflicting demands.

2.1. Conflicting Demands of Agile Distributed Development

Recent studies suggest that global software development requires both flexibility and rigor (Lee et al. 2006). Whereas flexibility can be achieved through agile development methods, the rigor required for distributed development is often offered by plan-driven approaches. Although each approach can be successful individually, pursuing them simultaneously poses several conflicts (Agerfalk and Fitzgerald 2006).

In distributed development, face-to-face meetings are very infrequent, or they tend to be sporadic, short, selective, and formal (Oshri et al. 2007). Communication frequency is dramatically decreased

when projects are distributed among multiple sites (Herbsleb and Mockus 2003, Matloff 2005, Meadows 1996). To minimize the resulting problems, such as the inability to maintain interpersonal ties and find appropriate information (Layman et al. 2006), impersonal and formal forms of communication using artifacts such as architecture plans, process specifications, and project plans (Hinds and Bailey 2000) are employed. Similarly, past literature suggests that distributed development relies on well-defined and formal processes and coordination techniques/tools to establish control. Typically, plan-driven approaches enhance control by a variety of strategies such as the development of a detailed architecture, heavy documentation, formal process-oriented tools such as workflow management systems and configuration management tools, formal reviews, and the use of ontology (Redmiles et al. 2007, Repenning et al. 2001, Wongthongtham et al. 2006). Moreover, building trust among members is challenging in distributed contexts (Handy 1995) due to information asymmetries and limited contact between members (Cramton 1997, Herbsleb and Mockus 2003, Jarvenpaa and Leidner 1998, Wiesenfeld et al. 1998). People at different sites are less likely to perceive themselves as part of the same team than people at the same site.

In contrast, agile development strongly emphasizes constant communication among team members and customers, particularly through face-to-face interaction (Highsmith and Cockburn 2001). For example, requirements engineering in agile development relies heavily on face-to-face communication over written specifications (Ramesh et al. 2007). Furthermore, agile development emphasizes “people” rather than “process”-oriented control (Cockburn and Highsmith 2001). Agile project management relies on leadership and collaboration instead of command and control (Highsmith 1999). Agile methods try to minimize the overhead associated with formal procedures and increase process agility by emphasizing interpersonal coordination. Self-organizing teams (Cockburn and Highsmith 2001, Lindvall et al. 2002) require team cohesiveness, trust among team members, and strong interpersonal relationships (Highsmith 1999).

In summary, distributed development relies on formal forms of communication, control, and shared documents (e.g., formalized design specifications), whereas agile development adopts informal communication and interpersonal collaboration. Although most software development projects may face the conflicting demands discussed above, their frequency and intensity are pronounced in agile distributed development. We draw from the literature on organizational ambidexterity to understand how these conflicting demands can be managed.

2.2. Organizational Ambidexterity

Organizational ambidexterity refers to the pursuit of conflicting demands on an organization that require trade-offs (Gupta et al. 2006). Structural ambidexterity refers to the creation of “dual structures” to separate conflicting demands into responsibilities of different organizational units. Contextual ambidexterity is the behavioral capacity to simultaneously pursue conflicting demands, such as the ability to demonstrate alignment and adaptability across a business unit. It is achieved by building a set of processes or systems that enable and encourage individuals to make their own judgments about how to divide their time between conflicting demands for alignment and adaptability rather than by creating dual structures (Tushman and O'Reilly 1996). A context does not dictate specific types of action; rather, it creates a supportive environment that inspires an individual to do “whatever it takes” to deliver results (Gibson and Birkinshaw 2004). Although ambidexterity was initially proposed at the organizational level, it has been conceptualized at the level of individual business unit, teams, and individuals as well (Andriopoulos and Lewis 2009, Gibson and Birkinshaw 2004, Mom et al. 2009, O'Reilly and Tushman 2004). Whereas agile and distributed development may be practiced in an organization through structural ambidexterity by separating projects that require either approach, contextual ambidexterity is required when agile and distributed development practices are needed within a single project. Therefore, we use contextual ambidexterity at the level of a development team as the theoretical lens in our research.

2.2.1. Conflicts Managed by Ambidexterity. Past literature on organizational ambidexterity has examined the simultaneous pursuit of various types of conflicting demands (Gibson and Birkinshaw 2004). These conflicts have been studied under various streams of literature (Raisch and Birkinshaw 2008), including organizational learning (March 1991, Rothaermel and Alexandre 2009), technological innovation (He and Wong 2004, Tushman and O'Reilly 1996), organizational adaptation (Adler et al. 1999), strategic management (Burgelman 1991), organizational design (Duncan 1976), and interorganizational relationships (Im and Rai 2008, Tiwana 2008). Here, we examine the conflicts between alignment and adaptability in bringing together agile and distributed development. Alignment involves stakeholders consistently working together, following established procedures, towards achieving the common goals of a business unit, whereas adaptability focuses on the ability to quickly meet changing demands in the task environment by reconfiguring processes (Gibson and Birkinshaw 2004). In agile distributed development

environments, simultaneous pursuit of alignment and adaptability involves achieving a balance between rigorous formal processes that emphasize prediction and control, and flexible processes that emphasize coordination and innovation.

2.2.2. Antecedents of Organizational Ambidexterity. The literature on organizational ambidexterity includes a mix of longitudinal, qualitative studies (Lubatkin et al. 2006) and cross-sectional, quantitative studies that provide insights on the links between organizational and environmental factors, organizational ambidexterity, and business performance (Gibson and Birkinshaw 2004, He and Wong 2004). Organizational context, which refers to the systems, processes, and beliefs that shape individual-level behavior in an organization, is considered a critical factor that leads to contextual ambidexterity (Gibson and Birkinshaw 2004). It is a combination of structural context, culture, and climate of a business unit. Structural context is defined as a set of administrative mechanisms that foster particular types of behaviors in employees. Whereas organizational culture includes the values, beliefs, and principles that are implicit and organizational practices that support these variables, organizational climate refers to the organizational and environmental characteristics that shape individual behavior. Gibson and Birkinshaw (2004) equate these variables of organizational context to discipline, stretch, support, and trust. Discipline induces members in a team to strive to meet all expectations generated by their explicit or implicit commitments (Gibson and Birkinshaw 2004) and can be described in terms of control, accountability, and structure (Andriopoulos and Lewis 2009). Stretch induces members to voluntarily strive for more ambitious objectives. Support refers to how members of an organization/team lend assistance to others. Trust induces members to rely on the commitments of each other. These four variables together constitute the “yin and yang of continuous self-renewal” of organizations (Ghoshal and Bartlett 1994, Gibson and Birkinshaw 2004) in that they are a combination of “hard” elements (discipline and stretch) that focus on performance and “soft” elements (support and trust) that focus on social cooperation (Gibson and Birkinshaw 2004). These four variables shape ambidexterity through the development of a suitable context that cultivates initiative, cooperation, and learning, leading to individual behavior, rather than institutionalization of organizational mandates. Thus, the combination of these variables shapes the organizational context that stimulates individuals to do what is necessary to address conflicting demands (Gibson and Birkinshaw 2004). In their empirical study, Gibson and Birkinshaw (2004) found that these variables collapsed into two factors—performance

management and social context. Whereas performance management includes discipline and stretch, social context is comprised of support and trust. These two factors have been further substantiated by recent research on senior team integration and organizational integration mechanisms (Jansen et al. 2009), specifically emphasizing the balance between formal and informal mechanisms in organizational integration. In our study, we recognize the role of performance management and social context in shaping ambidexterity, and examine how they are enabled in the context of agile distributed software development.

2.2.3. Outcomes of Ambidexterity. Ambidexterity has been long viewed as an important driver of long-term performance (Raisch and Birkinshaw 2008). Organizations that have been successful in dynamic environments have been viewed as ambidextrous, with a distinctive ability to balance current business and market needs and adapt to a changing environment for the future (Gibson and Birkinshaw 2004). Gibson and Birkinshaw (2004) argue that focusing on either short term (exploitation or alignment with current needs) or long term (exploration or search for new knowledge and radical innovation) at the expense of the other will result in tensions due to polarizing pressures. Organizations that concentrate on exploration without paying adequate attention to exploitation may invest in experimental innovations without developing the capability to implement and institutionalize them. In contrast, focusing on exploitation with limited or no exploration will result in inability to catch up to innovations that may be essential in dynamic environments (March 1991). Ambidexterity's role in shaping business unit performance is also highlighted by the choice of consistent profitability and ranking in the use of cutting-edge technologies as proxies of ambidexterity (Andriopoulos and Lewis 2009). Environmental dynamism, competitiveness, and firm's absorptive capacity have been found to moderate the relationship between exploration/exploitation and firm performance (Jansen et al. 2006, Rothaermel and Alexandre 2009). Cao et al. (2009) suggest that both balancing exploration and exploitation (referred to as balanced ambidexterity) and the magnitude of exploration and exploitation (referred to as combined ambidexterity) result in better firm performance. In examining the link between process management and ambidexterity, Benner and Tushman (2003) identify responsiveness and performance as critical outcomes of ambidexterity. In summary, past research on ambidexterity reports that ambidexterity leads to better outcomes measured by conventional financial metrics, responsiveness, or self-reported performance.

2.3. Ambidexterity in Software Development

In software development, ambidexterity has been treated as the ability to balance the tenets of different development methodologies (Vinekar et al. 2006). Agile distributed development poses conflicting demands between alignment and adaptability in that distribution requires stability, rigor, and discipline—consistent with the demands of alignment; and agile methods emphasize flexibility and responsiveness to change—consistent with the demands of adaptability. Achieving ambidexterity involves reconciling these conflicts. Whereas the literature on organizational ambidexterity identifies conflicting demands in several other contexts, to the best of our knowledge, ambidexterity in managing agile distributed development has not yet been studied. We address this gap by examining how software development organizations achieve ambidexterity by mitigating these conflicts.

The following three concepts identified in our literature review guided data collection and analysis:

1. *Conflicts*: This concept refers to the conflicting demands posed by alignment and adaptability in the context of agile distributed development.
2. *Project context*: This is comprised of performance management and social context.
 - a. *Performance management*: This concept, an important antecedent of ambidexterity, includes discipline and stretch, focusing on “hard” elements related to performance.
 - b. *Social context*: An antecedent of ambidexterity, this concept is considered as orthogonal to performance management in that it includes “socially oriented” elements of support and trust.
3. *Contextual ambidexterity*: This concept refers to the capability to pursue conflicting demands simultaneously.

3. Research Methodology

Because our research is exploratory rather than confirmatory, we use a case study design (Yin 2003). The lack of a systematic framework to understand ambidexterity in agile distributed development motivates the use of the case study approach to inductively develop such a framework (Eisenhardt 1989). Qualitative methods enable a researcher to delve into the complexity of problems in software development, and develop richer and more informative conclusions. Yin (2003) defines a case study as “an empirical inquiry that investigates a contemporary phenomenon within its real-life context, especially when the boundaries between phenomenon and context are not clearly evident.”

Empirically investigating ambidexterity in agile distributed development aligns well with the above description. Case study research is useful when

“a phenomenon is broad and complex, where the existing body of knowledge is insufficient to permit the posing of causal questions, when a holistic, in-depth investigation is needed, and when a phenomenon cannot be studied outside the context in which it occurs” (Paré 2004, p. 4). Our qualitative study follows the positivistic tradition. Orlikowski and Baroudi (1991, p. 9) explain that positivist research assumes “an objective physical and social world that exists independent of humans, and whose nature can be relatively unproblematically apprehended, characterized, and measured.” Also, positivistic case studies can be descriptive, exploratory, or explanatory. Our study is exploratory in that it focuses on inductive theory building. Although Eisenhardt (1989, p. 536) suggests that “theory-building research must begin as close as possible to the ideal of no theory under consideration and no hypotheses to test since preordained theoretical perspectives may bias and limit the findings,” she also acknowledges that it is nearly impossible to start with a clean theoretical slate. Paré (2004) argues that using elements of prior theory might be of “great help in focusing research efforts at the outset of the project” because it can provide a frame for analyzing the phenomenon that is central to the study. Our study uses constructs from the theory of organizational ambidexterity, which provides the initial framing necessary for developing an understanding of how organizations involved in agile distributed development balance conflicting demands. Also, consistent with Eisenhardt’s (1989) argument that early identification of possible constructs is tentative in theory-building research and that new factors will emerge during data collection, our data analysis process led us to identification of new factors that are used along with constructs identified a priori in the development of the conceptual framework.

3.1. Site Selection

Using theoretical sampling (Patton 1990), we selected three organizations and one project in each organization that had adapted agile practices for a distributed environment. These projects provided access to various team members who had rich experiences, both positive and negative, in agile distributed development, which provided rich data for the development of our framework.

To strengthen the generalizability of our findings, we used project teams from multiple sites to replicate the findings (Eisenhardt 1989, Miles and Huberman 1994, Yin 2003). The three sites are similar in their use of agile distributed development. Each had developed a plan-driven process with relatively formal procedures for distributed development. They also gradually implemented agile development practices in

projects that required that approach. They face market pressures to use distributed development, but create products that are required to meet evolving customer needs. Each project team, in its own assessment, considered that the use of balanced practices drawn from both plan-driven and agile approaches helped them meet conflicting demands and satisfy schedule, budget, and quality requirements. The project teams believed that their ability to develop balanced practices was critical to the success of their projects.

3.2. Data Collection

Qualitative data were collected from the three sites progressively via on-site and phone interviews and the review of documentation. Data collection and analysis were conducted in an iterative fashion over a period of 14 months. Table 1 provides details on the participants and company characteristics. We discussed our research goals with key contact persons who were senior managers in the three organizations. They helped identify participants who were intensively involved in agile distributed development (snowball sampling). In turn, these participants helped identify additional participants. We conducted semistructured interviews with each participant two or three times to collect data, seek clarifications, and obtain additional insights.

The interviews focused on a recent project that used agile distributed development. Each participant had worked on at least one prior project that used this approach. Data collection focused on the context in which agile distributed development was undertaken, the conflicting demands posed, and the practices that mitigated these conflicts. Each interview lasted from one to two hours. All the interviews were audio recorded and transcribed. Furthermore, additional data in the form of process documentation were used. Specifically, requirements documents, design

Table 1 Subjects in the Study and Company Characteristics

Role	Telco (T)	Manco (M)	Consult (C)	Total
Domain	Tele-communications	Supply chain	CRM	
No. of completed/ongoing agile distributed projects	6	4	3	
Number of participants				
Designer and developer (D)	4	4	5	13
Analysts and test engineer (T)	1	1	1	3
Project manager/lead (P)	2	2	1	5
Customer representative (C)	2	1	1	4
Total	9	8	8	25
Average experience of participants (in years)	10	9	10	

documents, and meeting minutes were reviewed to triangulate the qualitative data. Data collection was tightly interwoven with data analysis. Subsequent interviews were conducted to follow up on insights that emerged from data analysis. New data were used to refine and/or modify prior concepts and guide the next round of data collection. Data collection and coding were done until “theoretical saturation” (Eisenhardt 1989) was reached, when additional data did not add to the concepts and subcategories.

3.3. Data Analysis

Transcripts of interviews constitute the primary data in this study. First, interview data were coded to reflect constructs that were identified from our review of literature (conflicts, project context which includes performance management and social context, and ambidexterity, as described in §2) and the new constructs that emerged during our analysis. The new constructs related primarily to the balanced practices followed in the three agile distributed projects. Text segments in the interview transcripts ranging from short fragments to several sentences in length were tagged with one or more codes. We also noted the rationale for the assignment of codes. Finally, all the codes and notes were carefully examined to identify a set of major themes (Patton 1990) that form the framework presented in the next section. Atlas-ti® tool was used to organize and code the data. Through our analysis we identified key concepts, categories, and relationships from each project. Initially, two researchers separately analyzed the qualitative data from the sites. Then the analysis results were debated and modified until they reached agreement. The discovered theoretical concepts and relationships are included in the framework described in the next section.

The data analysis process provided multiple opportunities to establish plausibility and consistency of the concepts, subcategories, and relationships to ensure that our conceptual framework satisfies established criteria for the credibility and validity of qualitative research (Miles and Huberman 1994, Pawlowski and Robey 2004). No attempt is made to statistically evaluate the strength of concepts in the framework (Strauss and Corbin 1990). Theoretical relevance of concepts is indicated by their repeated presence or notable absence when comparing incidents (Strauss and Corbin 1990). Although the findings from such analyses may be detailed and particularistic, they can be used to develop a general explanation of the results (Eisenhardt 1989, Orlikowski 1993) through analytic generalization (Yin 2003). The research framework was presented to key informants, and on the basis of their feedback, minor adjustments were made to the terminology used. We present evidence

through quotes from qualitative data in accordance with recommended practice in qualitative research (Mason 2002). The organization and the role of the informant are also identified with a two-letter code representing the organization and the role of the informant (identified in Table 1). For example, MP represents a quote from a Manco (organization) project manager (role).

4. Findings

4.1. Case Study Sites

4.1.1. Site 1—Telco. Telco is a very large IS development organization of a utility company that employs over 2,000 IT professionals. As a part of a process improvement program, Telco adopted a standardized development process. It engaged a major consulting company to customize this standard process for distributed development. It further adapted this process for agile distributed development.

UP Project. The study participants were drawn from the UP project that developed systems that were mandated by the state public utilities commission and federal regulations. These mandates and the resulting system requirements were changing frequently. This project produced systems that supported information sharing between Telco and other service providers who shared the use of a telecommunications network.

4.1.2. Site 2—Manco. Manco is a large manufacturing company and its IS organization consisted of several hundred professionals. Recognizing the need to leverage offshore development, it had set up a center in India about three years prior to our study. To meet the demands of customers who favored an agile approach, the organization adapted several major agile development practices for use in distributed projects.

SC Project. The SC project extended the functionality of a complex supply chain system. Although the system had been in successful operation for several months, customers were demanding additional functionalities for developing ad hoc reports and what-if analysis. The requirements for this system evolved significantly during implementation because the users could not clearly articulate them at the outset.

4.1.3. Site 3—Consult. Consult is an established player in the field of IT-enabled services and employs several thousand IT professionals throughout the world. Its global delivery model using offshore development has been used successfully in hundreds of projects. It had dedicated a small facility exclusively for a U.S.-based customer with whom it was pursuing a significant long-term partnership. It was engaged in a few agile distributed projects for this customer.

CRM Project. The CRM project involved the development of a customer relationship management system to support a complex customer network. The end users, who were primarily sales and marketing professionals, favored an incremental approach to development. Early delivery of functionalities and a flexible approach to development were key requirements in this project.

4.1.4. Common Characteristics of Focal Projects.

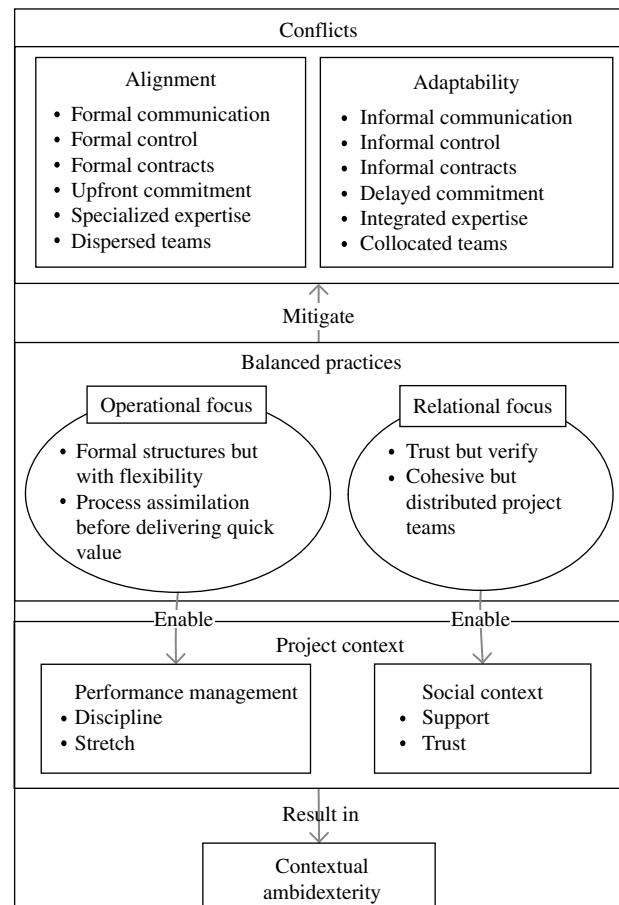
The need for distributed development was well recognized in each of the organizations. Whereas Telco and Manco are U.S.-based organizations that operated dedicated development centers in India, Consult had development centers dedicated to its clients both in India and the United States. Each of the organizations had significant experience in standardized processes for distributed development and in using multiple methodologies. Although the three organizations had successfully developed several systems using a plan-driven approach, they also recognized the need to be agile. In each project, customers were demanding a more flexible approach in which project scope and requirements were not frozen up-front, but were changing. Table 2 lists some of the salient characteristics of the projects studied.

4.2. Our Framework

We develop a conceptual framework (Figure 1) based on the analysis of data collected from the sites described above. This framework includes three key concepts that were synthesized from our review of literature (presented in §2): (1.) conflicts (between alignment and adaptability in the context of agile distributed development), (2.) project context (which includes two subcategories—performance management and social context, which are antecedents of contextual ambidexterity), and (3.) contextual ambidexterity. It also includes a fourth concept—balanced practices that emerged from our analysis of qualitative data. Subcategories of these four concepts also emerged from our data. Our framework is described in detail in this section and is briefly summarized as follows.

Conflicts between alignment and adaptability in agile distributed development are observed along six

Figure 1 Conceptual Framework



dimensions—(i) formal versus informal communication, (ii) formal versus informal control, (iii) formal versus informal contracts, (iv) up-front versus delayed commitment to requirements, (v) specialized versus integrated expertise, and (vi) dispersed versus colocated teams. To achieve alignment, formal communication, control, and contracts are needed. Requirements need to be specified explicitly up-front so that the development process can ensure that the developed system meets these requirements. Furthermore, the development teams are organized based on technical/domain expertise and locations to achieve operational efficiency. In contrast, to achieve adaptability, mechanisms that support rapid response to changes are necessary. For example, informal communication, control, and contracts are effective in providing quick response and feedback that are necessary to ensure that the development process meets the evolving organizational needs. Similarly, the development process should provide the flexibility to accommodate changes in user requirements rather than requiring commitment to a fixed set of requirements. Finally, an integrated and colocated team is necessary to enable adaptability. These conflicts are *mitigated by* the balanced practices that fall into two groups—operational

Table 2 Summary of Project Characteristics

Company	Average project duration (months)	Application complexity	No. of development sites	No. of customer locations	Average development team size (people)
Telco	6	Moderate	2	2	16
Manco	10	High	3	5	14
Consult	7	Moderate	3	3	15

and relational. Operational elements focus primarily on quality, productivity, and efficiency, whereas relational elements focus on closeness among the stakeholders involved (Stank et al. 1999). We classify the first group of practices (formal structures, but with flexibility, process assimilation before delivering quick value) as oriented towards achieving balance in the operational aspects of the development process, and the second group of practices (trust but verify, cohesive but distributed project teams) as oriented towards achieving a balance in the relational aspects of the development process. These two groups of balanced practices, in turn, enable two groups of antecedents of contextual ambidexterity. Whereas the operationally focused balanced practices enable performance management (discipline and stretch), the relationally focused balanced practices enable social context (support and trust). Thus, balanced practices lead to contextual ambidexterity by mitigating the conflicts between alignment and adaptability simultaneously. Our framework is explained in detail in the following subsections.

4.3. Conflicts Posed by Agile Distributed Development

The three project teams faced tension between the demands of distributed development and agile development in the following areas. We present each of these conflicts in terms of dimensions of alignment and adaptability.

- Alignment vs. adaptability in communication.

Formal vs. informal communication: In distributed development, the three organizations often relied on formal mechanisms (such as detailed architectural designs and plans) to address impediments to communication that resulted from geographical separation. “We realized that it is not easy to work with people who are not next door to you. You had to collaborate, sometimes very intensely, with developers across the globe and it did create some problems” (MT), claims a test engineer at Manco.

In contrast, agile development required more “informal interactions [rather] than explicit documentation” (MT), which is difficult to achieve in a distributed environment. Consult recognized the “need to have face-to-face time with the customers and developers” (CD) and set up a dedicated center in the United States for its client.

- Alignment vs. adaptability in control.

Formal vs. informal control: In distributed development, the organizations relied on “formal processes for establishing control” (MP). Agile development, on the other hand, is more “people oriented and control is achieved informally” (MP). A project manager in Manco states that “We try very hard to keep the focus on developing the product that the customer wants

in a timely fashion, and we will give up some of the formalized controls we use in our organization if it is necessary to meet this overarching objective” (MP).

This created a dilemma for the organizations in balancing people- and process-oriented control in agile distributed development.

- Alignment vs. adaptability in contracts.

Formal vs. informal contracts: Contracts in agile development are “loosely and informally defined” (CP). For example, the top management at Consult was willing to bend over backwards even if it meant significantly changing its well-established, standardized global delivery process. “The project is really ill-defined for us to use the standard process anyway, so we have been directed to tailor the process as much as necessary” (CD), explained a lead developer.

In this project “the customer is always right, even if they now want something different from what is stated in the documents” (CC). In contrast, distributed development in this team relies on “explicit targets, milestones, and detailed specification of requirements” (CD). Therefore, there is a need to balance the level of formality in developing contractual agreements.

- Alignment vs. adaptability in commitment to requirements.

Up-front commitment vs. delayed commitment: In distributed development, the organizations relied on “fixed, up-front commitments on quality requirements” (MC) because it is much easier to manage remotely located teams if there are “well defined expectations” (MC). In contrast, agile development relies on ongoing negotiations between developers and customers for determining “acceptable levels of quality” (MD) at various stages of development. In all the three project teams, the customer requirements were evolving rapidly and nearly 50% to 60% percent of the requirements changed during development. In Telco, the “customer requirements were not at all stable. In fact, much of these requirements were based on interpretations of the legal requirements imposed by regulators. Frequently, customers changed their interpretations and therefore what the system had to do to meet these requirements” (TT). In the SC project, “one of the most important considerations was meeting the customer requirements, no matter how often they changed. The project changed quite a bit before the requirements got stabilized” (MT). Thus, there was a need to handle the tension between up-front and delayed commitment to requirements had to be developed.

- Alignment vs. adaptability in expertise integration.

Specialized vs. integrated expertise: Distributed development typically organizes domain and technical expertise into specialized subgroups. For example,

application domain experts are often located onshore at client sites, whereas technical experts are located offshore. In Manco, “technical teams are 9 time zones away from the clients who are the real experts in the area [application domain]” (MT). In Telco, “understanding the regulations requires deep knowledge of the industry that we didn’t initially have” (TD), stated a developer. Such specialization makes it difficult for project participants to bridge differences in expertise. In contrast, agile development typically requires a team to possess both domain and technical expertise that are needed for successful development. Whereas project subgroups are separated based on expertise in distributed development, they are integrated in an agile project team.

- Alignment vs. adaptability in team cohesion.

Dispersed vs. colocated teams: Colocated teams find it is easier to achieve “team cohesiveness and shared view of goals” (TD), which helps them “readily react to the changes in the user needs and preferences” (TD), explained a Telco developer. “With agile development, we need a team which works together with a single purpose. When the team is dispersed across multiple sites, it is difficult to achieve [adaptability]” (CP), explained a project manager at Consult. The three project teams strived to develop practices such that team cohesion can be improved within the constraints of a distributed environment.

4.4. Balanced Practices

Project teams in the three organizations developed innovative practices that were effective in mitigating the conflicts described above. These practices, which may be characterized as agile but disciplined, have evolved in the three teams with repeated experimentation. The balanced practices described below are in the spirit of agile methods, but have been adapted to meet the demands of distributed development as well. Although the practices presented here were observed in our study, we do not claim that we have developed an exhaustive list of all practices that lead to ambidexterity. We have classified the observed practices into four groups. Two of these have an operational focus and two have relational focus.

4.4.1. Practices with Operational Focus.

1. **Formal Structures but with Flexibility:** The organizations adopted several ways to balance the formality and flexibility needed in the focal projects. Instead of strictly following agile development practices as commonly defined, the three organizations continuously tweaked them to fit the evolving needs of their projects.

a. *Formal channels to facilitate informal communication:* There were several incidents of miscommunication among the members of the distributed teams in Manco. These were attributed to “overreliance on

informal communication” (MC). To minimize this problem, a lead developer was designated as the primary contact person for each location. Telco valued “*formal channels*” (TT) for communication. Therefore, the use of a lead developer as a “gate keeper” (TT) was welcomed by senior management. This gate keeper helped “to make sure at least one person knew exactly what is being communicated on either side and that nothing falls through the cracks when so many people are interacting” (TT). Thus, in both organizations, informal communication among the team members was facilitated through formal channels.

b. *Formal but limited coordination:* Whereas typical agile development relies on minimal coordination of the team’s activities by project managers, in all three organizations, the “coordination” roles of project managers/leads were significantly more important. In Telco, the manager maintained a “formal structure of reporting and responsibility” (TP). Specifically, the lead developer and project champions were expected to coordinate the activities of the offshore and onshore teams to help achieve project goals. “These individuals make sure that project members in each side are able to go to one individual who can coordinate with the counterpart on the other side to get things done” (TP).

c. *Plan iterations to finalize requirements and develop designs:* The project managers at both Manco and Consult were skeptical of agile methods that do not include adequate up-front design. In both organizations, “the first two or three iterations of a project were devoted to finalizing critical requirements and developing a high-level architecture” (MD). Specifically, they used a SCRUM-like process for “two up-front design cycles” (CD) to help finalize requirements. Similarly, a lead developer at Telco characterized the project as follows: “The end users in this project are not in a position to clearly articulate what they need in the system. They want us to work with them and show them the kinds of things that can be implemented. Then, they are able to understand what is feasible and state other capabilities they need” (TD).

d. *Document requirements at different levels of formality:* Telco and Manco changed their requirements analysis processes significantly for agile distributed development. Telco used “short use cases and user stories instead of detailed use case specifications” (CD). Also, product managers and customer representatives were encouraged to clarify requirements by sketching out acceptance tests. At Telco, the members of the offshore team felt that such “minimally documented requirements were more helpful than relying on just informal communication” (TT). At Manco, “we want to create a culture of experimentation and innovation in product development... The project

enjoys a high profile because we are directly working with some senior managers in the company. Since they are at the center of this development, we need to have a flexible process that accommodates their schedule and needs, but delivers outputs quickly" (MP), explains the project manager.

This group of practices addresses two conflicts: "formal versus informal communication" and "formal versus informal control." The conflict "formal versus informal communication" is addressed by maintaining communication through multiple channels and limited structure. "Though we want the customer to be able to tell the developers at any time about the changes that they [want] implemented, we also need to keep some control over the process. So, we ask the customer to initiate a change request, even if it is after the developer has started working on the change" (MP) states a project manager at ManCo. The conflict "formal versus informal control" is mitigated by ensuring that the process is just right for the current needs. "We give the developer the flexibility necessary to meet their goals. We have given them flexibility to decide on the amount of formal reporting and documentation that is appropriate for different situations" (TP), states a project manager at Telco.

2. Process Assimilation Before Delivering Quick Value: All three organizations emphasized the importance of developers gaining a good understanding of the application and business domains. Processes and tools were developed to minimize the overhead to participants in knowledge sharing.

a. *Focus on well-understood functionality rather than "critical" new functionality:* A project manager at Manco initially wanted to "build an atmosphere in which both the developers and the client representatives were comfortable with the new processes, tools, and the application" (MP). Therefore, the team was tasked to work on well-understood functionalities in the early iterations rather than on complex requirements even though "some were prioritized by the customers as more critical" (MP). The customer representative also agreed with this approach because it "helped (him) understand the development environment better" (MC). Although this approach is against the spirit of agile methods, which emphasizes early delivery of value to the customer by focusing on critical features, it was necessary to "achieve a level of knowledge sharing that is essential for project success" (MC).

b. *Variable length but short cycles:* A common characteristic of most agile development projects is time-boxed development. Customers in all three organizations had a strong preference for incremental development of capabilities in successive releases. Telco customers demanded that "new features must be deployed quickly. Otherwise, the company may be out of compliance with the regulations, which can

cause some serious problems" (TC). Senior executives who were the customers in Manco had "very little patience for long delays in the development process" (MC). Their attitude was that "you have all the resources both locally and in the offshore center and can literally work 24×7 to deliver what we need" (MD), explained a developer. The customer representative at Consult said that "the close involvement with the customer and the ability to provide them 'software candy' periodically in the form of new features" (CC) was a key project requirement. Telco and Consult realized that this approach, although successful with their colocated teams, "was not working in distributed development" (CC). In Telco, early iterations in three previous projects faced difficulties because the "development team did not develop sufficient understanding of the business needs" (TT) of the customers. In Consult, critical requirements provided by the clients in the first iteration were "too broad in scope" (CC) to be implemented within a short time-boxed iteration. Both of the organizations adopted a flexible short-cycle approach in which two to three early development cycles were allowed to take two to four weeks each, depending on the complexity and the "set-up" time needed to understand the business domain.

This group of practices helps developers gain a thorough understanding of the business domain and user requirements and thereby addresses the conflicts "formal versus informal communication" and "formal versus informal contract." "It is important for us to be on the same page of music. Understanding exactly what the customer wants at the present time is more important than what was stated in a document at the beginning of the project. Therefore, my team is encouraged to directly talk to the customer at any point and make changes that the customer wants without waiting for written change requests" (CP), states a project lead at Consult.

4.4.2. Practices with Relational Focus.

3. Trust but Verify: Deviating from agile methods, the companies introduced several practices to formally verify the quality of the processes used by the offshore teams while gradually building trust.

a. *Trust the development team, but manage quality:* All organizations worked on building trust between offshore and onshore teams by achieving shared understanding and agreement on the adopted agile processes. Early delivery of features facilitated trust building within the teams. However, the teams also adopted mechanisms that helped verify that the processes followed would deliver high-quality output. Senior management at Manco insisted that the onshore QA team be involved in ensuring that the "practices followed by the offshore development team were of acceptable quality" (MT). In each iteration, this QA

team reviewed the test procedures created by the off-shore development team. In Telco, the onshore technical personnel participated in design reviews to ensure that the offshore development met their quality standards. Such practices helped establish an “adequate level of checks and balances in the process while trusting the participants to follow due diligence in their work” (TT).

b. *Develop through informal collaboration, but document post facto for verification*: In the spirit of agile development, even Telco, which “favors a formal structure” (TP), cultivated an informal atmosphere to facilitate collaboration. However, the management insisted on supplementing this with “documentation of critical artifacts” (TP). Strangely enough, often many documents were developed “after the actual work was completed” (TD). For example, requirements documents were finalized towards the end of the development cycles rather than at the beginning so that they “represent the actually implemented functionality rather than the initial requirements” (TC). A somewhat analogous practice was also considered necessary at Consult in order to formalize the “*vendor-client [contractual] relationship*” (CC).

This group of practices developed trust but also included verification procedures to manage the quality of processes, thereby addressing the conflict “upfront versus delayed commitment.” “While we were committed to satisfying the evolving requirements of the customer, we also documented the justifications for the requested changes as soon as possible” (CD). This group of practices also addresses the conflict “formal versus informal control” by establishing a level of control essential to manage the quality of the development processes while providing necessary flexibility. “While the development team was encouraged to make changes as and when suggested by the customer, they were subject to subsequent review by the project manager” (CP).

4. Cohesive but Distributed Project Teams: The organizations realized that team cohesiveness is essential in agile distributed development because their projects used minimal formal controls. Several practices were used to establish cohesiveness among team members.

a. *Synchronized work hours for relationship building*: Although 24×7 development is one of the stated goals of the global delivery model, this was far from reality in the three organizations. Separation by over 9 to 12 time zones created serious bottlenecks to achieving effective communication and relationship building. Whereas many aspects of distributed development can be done with asynchronous collaboration, this was not consistent with constant communication needed for agile development. To “share the pain” (TP) of synchronized work schedules, all of the

three organizations used “early morning meetings for onshore customer representatives and product managers and late evening shifts for offshore developers” (CP). “This way we could ensure that all the team members, both onshore and offshore were at work at least for a part of their work day at the same time. This overlap helped them interact with each other, for example, to share deliverables, assess project status, share concerns, resolve issues together” (CP).

b. *Constant communication to establish team cohesiveness*: A variety of mechanisms were used in each organization to maintain constant communication to build a cohesive team. At Telco, short morning meetings were held at the beginning of each work day to “identify issues, track project status, and invite ideas and critiques” (TT). Also, members used online chat and Short Message Service/text messaging extensively. Lead developers and project champions were almost on a “ 24×7 call” (TP) on their Blackberries® at Consult. The “instant” availability, while considered beneficial, also was a significant burden on these “*gate keepers*” (TT) of communication. Furthermore, at both Telco and Manco, senior managers used video conferencing on a weekly basis to initiate new development cycles, assess progress at the end of each cycle, and discuss critical issues.

c. *Enable team cohesion with frequent visits*: In all three organizations, customers or surrogates (product managers) visited the development team on a regular basis to establish a close working relationship. These visits effectively reduced the impact of distribution on team cohesiveness. Consult had a dedicated facility at its customer location, which was staffed with a “small group of analysts and developers” (CD). Telco and Manco organized visits to customer sites by developer representatives from offshore development teams. In Manco, many long visits (3 to 6 weeks) were used to transfer domain knowledge and develop trust and team cohesion. In Telco, key customer representatives spent several weeks in their offshore development center. These visits were more intense “both during early development cycles and when the project was implementing major changes in requirements” (TC). Senior managers and onshore teams also visited the offshore development teams at the beginning of the project to “create a close and informal working relationships at all levels” (MP), in addition to “establish[ing] ground rules and finalizing contractual arrangements” (CP), and evaluate project progress at the end of critical milestones. More importantly, these visits helped establish trust and create personal relationships that mitigated miscommunication and misunderstanding.

d. *Maintain and nurture working relationships*: Telco created a cohesive team culture by requiring that each team consist of members who had

developed “prior working relationships with each other” (TP) and collectively possessed all of the required expertise. The management insisted on such an arrangement to minimize what they perceived as “chaos” (TP) in agile development. Consult adopted a similar strategy and created “high performance” (CP) teams made up of very experienced personnel. In all of the three organizations, the tremendous demand for well-qualified developers posed a significant challenge in retaining a cohesive team. They tried to keep teams together over multiple projects instead of building a new team from scratch.

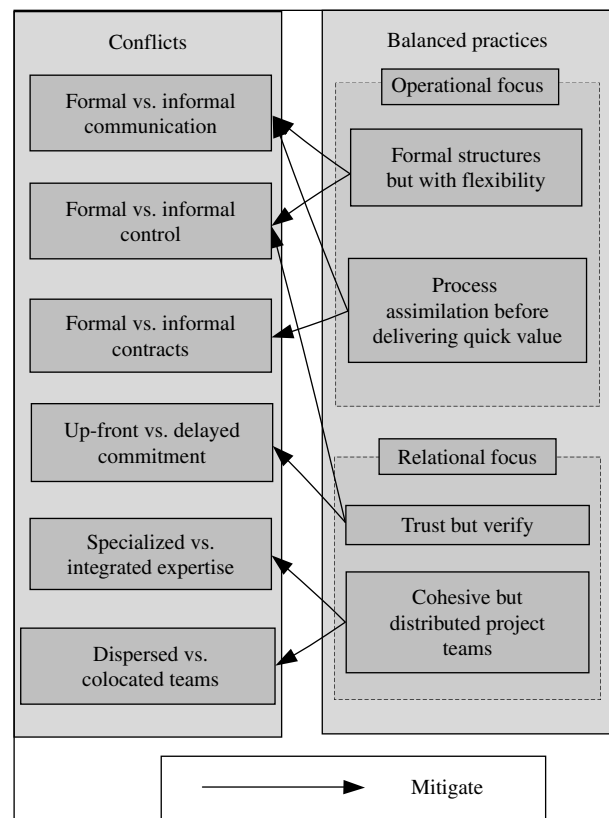
This group of practices mitigates the conflict “dispersed versus colocated teams” as it tries to minimize the impact of distribution on the development of trust between the teams. “Since the team is made up of people who know each other well and have in some cases a long history with each other, they can work as a single unit even though at times they may be working in different locations” (TP), explained a manager at Telco. This group of practices also addresses the conflict “specialized versus integrated expertise” by bringing together domain and technical expertise. “We encouraged close collaboration between the customer, developers, and other project personnel in as many ways as possible so that they can operate with a common purpose” (MP).

In summary, we have identified four groups of balanced practices that enable alignment and adaptability simultaneously by mitigating the conflicts in agile distributed development. Although each group can mitigate any of the conflicts presented to varying degrees (that is, a many-to-many relationship is conceivable between the various conflicts and balanced practices), we only provide some illustrations of each group mitigating conflicting demands that were observed in the focal organizations (Figure 2).

4.5. Project Context

4.5.1. Performance Management. The balanced practices enable the key antecedents of ambidexterity, viz., performance management and social context, which include discipline, stretch, support, and trust. In Manco, the team leader’s assessment of how the team functioned indicates strong discipline that involves members in the team working to meet all expectations generated by their explicit or implicit commitments: “We work as a team with a common goal of making this project a success. Each person does what is expected of him and delivers what is promised, without fail” (MP). In agile distributed development, discipline is associated with the balanced practices that imbue process-oriented control while providing an environment that enables open but managed communication. The three project teams

Figure 2 Mapping Between Conflicts and Balanced Practices



balanced process control with intensive communication to provide discipline. “We rely on frequent communication rather than on formal reviews to keep the project in control. We seem to be on a 24 × 7 clock because there is so much communication back and forth among the team members spread across several locations” (CP). This helped them achieve open, candid, and rapid feedback.

In the three projects, stretch was quite evident in that team members voluntarily strive for more ambitious objectives. For example, at Consult: “This project is very important to establish a long-term relationship with the customer. All of us try very hard to go well beyond the expectations of the customer and make the customer happy at any cost” (CD). All the three project teams achieved stretch by focusing on common goals across distributed teams. “We work as a coherent team, and each one of us has a feeling of ownership because we know our contributions are valued. The common purpose of the project has been clearly communicated to each member of this effort and has their buy-in. We strive hard to make sure that we don’t lose sight of this common purpose” (MD). This helped them establish shared goals and collective identity. Thus, the group of practices that have an “operational focus” (formal structures but with flexibility, process assimilation before delivering

quick value) helps establish process-oriented expectations while providing an organizational context that fosters productivity and efficiency, thus enabling performance management (discipline and stretch).

4.5.2. Social Context. Support was evident in the three project teams in that team members readily lent assistance to each other. “We all pitch in for each other. Since we all have a common goal, we need to help each other to achieve that goal” (TP), stated a project manager at Telco. Support was achieved by establishing a balance between people- and process-oriented control. “The developers have tremendous flexibility in performing their tasks, as long as they satisfy the customer. I really don’t care or even want to know in detail how each member delivers what they are tasked to do. I know from their track record that I can rely on each of my team members to produce what is expected of them, no matter what it takes” (TP), states a Telco manager. The ability to access all the available resources and a supportive management helped in this process.

The presence of trust was also clear in that team members relied on the commitments of each other. “I can be absolutely sure that if someone promises to deliver something, simple or difficult, it will be delivered. It is important to have such a high degree of trust in my team so that I can deliver on what I promise to the clients” (CP), states a project manager at Consult. Trust is built through the balance between team member commitment and formal measures. “We have checks and balances [with] informal reviews and documentation to ensure that the customer needs are satisfied, but we keep them to the minimum. While my trust of my team is absolute, it is important to keep the customer happy by making sure that they get to see what we have achieved through some form of documentation that they are comfortable with as long as it does not become a burden that slows us down” (CP). Thus, the group of practices that have a “relational focus” (trust but verify, cohesive but distributed teams) helps in relationship building by increasing trust and cohesiveness of software development teams through carefully shaping social context (support and trust) among the project team members involved.

4.6. Contextual Ambidexterity

Our findings provide insights into how ambidexterity can be achieved in the context of agile distributed development. The balanced practices presented in §4.4 enable performance management and social context, which in turn result in contextual ambidexterity. The three projects used a mix of formal and informal communication to achieve ambidexterity. Conflicts in control were managed through maintaining a considerable amount of informality to enable flexible development while putting in place processes that were

just enough to verify the practices followed by off-shore teams. Further, the three organizations also balanced the need to have formal contracts with the need to allow the development to adapt to changing organizational needs. The three project teams used several mechanisms to assure the quality of the product and meeting the customers’ requirements. While critical changes were documented to make sure they are tracked and implemented, this was considered secondary to moving forward with development. In the three projects, frequent visits by distributed partners and sponsors (for gaining trust and enhancing team cohesion) resulted in a good balance between trust and formal contracts and bridged gaps in expertise between domain experts and development personnel.

5. Discussion

Our study addresses the research question presented in §1 by describing how organizations achieve ambidexterity through the balanced practices that mitigate conflicting demands and enable performance management and social context. Past literature suggests that metaroutines can be used to systematize nonroutine processes so that a balance between innovative and routine processes can be achieved, resulting in ambidexterity (Adler et al. 1999). In agile distributed development, the balanced practices presented here can be used to develop such metaroutines. Adler et al. (1999) suggest either switching between tasks that pose conflicting demands or partitioning the organization into subunits that focus on individual (conflicting) demands. Switching and partitioning are not feasible in agile distributed development. Although structural ambidexterity may be achieved by separating agile and distributed development across different organizational units, this may not be appropriate in situations where a single organizational unit (such as a project team) needs to simultaneously pursue both agile and distributed development. In such contexts, we describe how software development organizations achieve contextual ambidexterity by using processes and systems that enable the simultaneous pursuit of alignment and adaptability within a project team. Also, past research has not identified specific antecedents of ambidexterity in the context of agile distributed software development. Our research addresses this gap by identifying specific balanced practices that provide insights into what constitutes ambidexterity in this increasingly important context faced by IS organizations. It also highlights how these balanced practices enable the antecedents of performance management and social context, which in turn result in contextual ambidexterity. These practices focus on behavioral orientation towards mixed capacities rather than

separation of concerns. Prior research suggests that systems and processes need to be developed to enable individuals to focus on whatever it takes to balance the conflicting demands (Gibson and Birkinshaw 2004). Our study answers this call by developing practices that guide the development of processes that foster both agile and distributed development.

Although past literature has highlighted the role of performance management and social context in shaping ambidexterity (Gibson and Birkinshaw 2004), there has been a paucity of prescriptions that help improve performance management and social context in the context of agile distributed development. We address this gap by identifying specific practices that shape performance management and social context. The links between the antecedents of contextual ambidexterity and the two groups of balanced practices presented here can also be substantiated by drawing from past literature. For example, these antecedents are consistent with the view of organizational context discussed in prior literature on organization science (Ghoshal and Bartlett 1994, Gibson and Birkinshaw 2004). Organizational context comprises of structural context, organizational climate, and organizational culture (Gibson and Birkinshaw 2004). This presents organizational practices in a spectrum of tangible versus intangible processes. The tangible processes enable organizational performance variables, whereas intangible processes enable trust and cohesion among project team members.

The two categories—performance-oriented and socially oriented elements—of organizational context map well to the two groups of practices that we have identified. Whereas the first group (formal structures but with flexibility, process assimilation before delivering quick value) centers primarily on managing performance of the software development teams, the second group (trust but verify, cohesive but distributed project teams) centers on socially oriented elements of the development process such as trust and cohesion in teams. Performance management—which includes discipline and stretch—is enabled by fast-cycle feedback and the development of collective identity (Ghoshal and Bartlett 1994). This is consistent with the link between the first group of practices and performance management. This group provides a context that will enable fast-cycle feedback and collective identity through appropriate mechanisms that formalize channels for communication, control, and documentation. The link between the first group and performance management is also consistent with the view that discipline focuses on behavioral outcomes shaped by control mechanisms, rather than the exercise of control (Ghoshal and Bartlett 1994). Social context—which includes support and trust—is enabled by factors such as involvement, autonomy,

and providing guidance and help rather than focusing on authority (Ghoshal and Bartlett 1994). This supports the finding from our study that links the second group of practices to social context. The second group develops cohesion among teams and provides mechanisms that will help project team members verify the quality of processes and deliverables while developing trust. Past literature on organizational context and ambidexterity substantiates the connection between the empirically grounded practices identified in our research and the theoretically grounded concepts associated with contextual ambidexterity.

Our results are also consistent with and supplement findings from prior empirical studies on distributed and agile development. For example, in a study of global, distributed development at IBM, Ambler and Nizami (2007) observe that the greatest challenge in integrating quality management in distributed development is posed by communication barriers due to distance, time, and cultural differences. Our findings provide guidelines to overcoming these communication barriers. For example, the “formal structures but with flexibility” group provides several practices that enhance communication between geographically distributed teams. In addition, our findings also point out that verification of processes is critical in quality management. Miller (2008) not only identifies practices that worked effectively at Microsoft, but also warns against the use of certain practices in agile distributed development. For example, he cautions against reorganizing teams for each new project. This is supported by our framework, which highlights social context as an antecedent of ambidexterity and emphasizes the importance of maintaining and nurturing working relationships. Miller (2008) also recognizes the need to adapt agile practices to include more documentation and collaboration tools in agile distributed projects. Our study complements this finding by suggesting a variety of formal structures that also provide flexibility.

6. Contributions

6.1. Contributions to Theory

Findings from our study contribute to the understanding of ambidexterity in the context of agile distributed software development. Our study is one of the first attempts at conceptualizing ambidexterity in the context of incorporating agile methods in distributed software development. Our research contributes to two theoretical areas:

6.1.1. Software Development. Much of the work in agile development has been either inspired by industry practices or by practical considerations in implementing them. This has resulted in paucity of theoretically grounded research in this area. Although

there has been some recent work linking theories from organizational and management literature to agile practices (Slaughter et al. 2006), little attention has been paid to developing such links to practices that balance conflicts posed by agile distributed development. Our research is one of the first attempts at connecting balanced practices identified empirically to the theory of ambidexterity. Although the development of individual practices that address various challenges in software development and the evaluation of these practices in controlled contexts is useful, linking such practices to elements of theories that have been well-established results in a more substantive set of findings (Orlikowski 1993). In a similar spirit, we link the balanced practices identified here to specific antecedents of organizational ambidexterity. Our research provides the initial step in the development of the much-needed theoretical basis for understanding and prescribing practices that balance agile and distributed software development.

6.1.2. Ambidexterity. This research takes a step in establishing the nature of ambidexterity in software development. Prior attempts at linking ambidexterity to software development (Lee et al. 2006, Vinekar et al. 2006) establish the need for agile and traditional approaches to coexist. However, there has been limited research on understanding how organizations can develop ambidextrous capabilities to pursue agile and distributed development simultaneously. Here we develop the notion of ambidexterity in software development as the ability to manage conflicting demands when there is a need to use agile practices in distributed development. Much of the literature on ambidexterity has centered on identifying antecedents of organizational ambidexterity, proposing ambidexterity as a variable that mediates the relationship between antecedents and various organizational performance measures. We further extend this work by proposing the practices that impact the antecedents of ambidexterity. In summary, this research provides an initial understanding of *what constitutes ambidexterity in agile distributed software development and how this capability may be achieved*.

6.2. Contributions to Practice

The practices identified in this research, directed towards top management, project managers, developers, and tool vendors, constitute concrete prescriptions to develop contextual ambidexterity.

6.2.1. Top Management. We suggest that the top management provide a suitable environment for the project to foster ambidexterity. Instead of enforcing formal top-down project control, top management should empower the project team to use the balanced practices to facilitate ambidexterity. Incentive structures provided to developers need to be aligned with

the ability to meet conflicting demands. Also, our findings indicate the need to develop mixed capacities to simultaneously pursue agile and distributed development rather than creating dual structures that separate the two types of development. Top management should refrain from creating structural separation of agile and distributed development, and encourage project managers and developers to adopt balanced practices.

6.2.2. Project Managers. Specific practices described in our framework represent concrete prescriptions for project managers. Project managers should adapt these practices further based on contextual factors. For example, for projects that have low levels of requirement volatility, more-formal requirements documentation may be more effective. Also, project managers who have been long entrenched in conventional, plan-driven development may resist relaxing formal requirements. However, to develop ambidexterity, they should recognize the importance of balanced practices such as supporting intense informal communication among developers while moderating them through formal channels.

6.2.3. Developers. Our study emphasizes the criticality of mixed capacities rather than separation of concerns across organizational units. Balanced practices help developers achieve this objective. Rather than implementing distributed development practices for one task and adopting agile practices for a different task (i.e., task-based differentiation to achieve a balance), developers should focus on achieving a balance between these practices. For example, let us consider the tension between informal and formal communication. Instead of focusing on using formal channels for some tasks and informal channels for others, developers should adopt tools that enable an appropriate mix by enhancing informal communication while supplementing it with “just enough” documentation.

6.2.4. Tool Vendors. Because the need for tool support for agile distributed projects is considerably different from support provided by existing tools, vendors of work process and productivity tools (e.g., communication, collaboration, modeling tools) need to support the needs of balanced practices. For instance, commonly used communication tools that facilitate informal communication can be enhanced with support to capture “just enough” documentation. For example, a video conference tool that is used for intense informal communication should be incorporated with the ability to capture episodes of communication and seamlessly link them with documentation created by developers.

6.3. Limitations and Future Research

Generalizations from our study should be made with caution in light of its limitations. Although our study encompassed three different projects and stakeholders at different levels in the organizational structure, our findings may be particularistic to these teams. This suggests further empirical studies to examine the validity of these practices in different organizational contexts. Also, although we develop the link between practices and contextual ambidexterity, further research is needed to establish the link between contextual ambidexterity and performance. Although past research has established this link in a wide variety of other contexts, further research is needed to examine the performance measures that are appropriate in the context of agile distributed development. Among the commonly used measures of outcomes of ambidexterity, adaptation, which includes responsiveness and performance (Benner and Tushman 2003), emerges as the most relevant for agile distributed development. Benner and Tushman (2003) suggest that the nature of the environment (stable or turbulent) and ambidextrous organizational forms moderate the relationship between exploitation/exploration and performance and responsiveness. Because agile distributed environments present a unique blend of stability and turbulence, a combination of traditional performance metrics and responsiveness may be warranted. Whereas traditional software development projects are typically assessed using the classical triple constraints of cost, quality, and schedule, agile distributed development needs to be assessed on responsiveness as well. Finally, our study provides a fertile ground for a large-scale quantitative study operationalizing the various elements in our framework and examining the relationships among them, including relevant performance metrics.

References

- Adler, P. S., B. Goldoftas, D. I. Levine. 1999. Flexibility versus efficiency? A case study of model changeovers in the Toyota production system. *Organ. Sci.* **10**(1) 43–68.
- Agerfalk, P. J., B. Fitzgerald. 2006. Flexible and distributed software processes: Old petunias in new bowls? *Comm. ACM* **49**(10) 27–34.
- Ambler, S. W., K. Nizami. 2007. Agile strategies for geographically distributed quality management. *Agile J.* <http://www.agilejournal.com/articles/columns/column-articles/720-agile-strategies-for-geographically-distributed-quality-management>.
- Andriopoulos, C., M. W. Lewis. 2009. Exploitation-exploration tensions and organizational ambidexterity: Managing paradoxes of innovation. *Organ. Sci.* **20**(4) 696–717.
- Benner, M. J., M. L. Tushman. 2003. Exploitation, exploration, and process management: The productivity dilemma revisited. *Acad. Management Rev.* **28**(2) 238–256.
- Boehm, B., R. Turner. 2003. *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley, Boston.
- Burgelman, R. A. 1991. Intra-organizational ecology of strategy making and organizational adaptation: Theory and field research. *Organ. Sci.* **2**(3) 239–262.
- Cao, Q., E. Gedajlovic, H. Zhang. 2009. Unpacking organizational ambidexterity: Dimensions, contingencies, and synergistic effects. *Organ. Sci.* **20**(4) 781–796.
- Cockburn, A., J. Highsmith. 2001. Agile software development: The people factor. *IEEE Comput.* **34**(11) 131–133.
- Cramton, C. D. 1997. Information problems in dispersed teams. *Proc. Annual Meeting of the Academy of Management, Boston*.
- Duncan, R. 1976. The ambidextrous organization: Designing dual structures for innovation. R. H. Killman, L. R. Pondy, D. Slevin, eds. *The Management of Organization Design*. North-Holland, New York, 167–188.
- Eisenhardt, K. M. 1989. Building theories from case study research. *Acad. Management Rev.* **14**(4) 532–550.
- Ghoshal, S., C. A. Bartlett. 1994. Linking organizational context and managerial action: The dimensions of quality of management. *Strategic Management J.* **15**(5) 91–112.
- Gibson, C. B., J. Birkinshaw. 2004. The antecedents, consequences, and mediating role of organizational ambidexterity. *Acad. Management J.* **47**(2) 209.
- Gupta, A. K., K. G. Smith, C. E. Shalley. 2006. The interplay between exploration and exploitation. *Acad. Management J.* **49**(4) 693–706.
- Handy, C. 1995. Trust and the virtual organization. *Harvard Bus. Rev.* **73**(3) 40–50.
- He, Z.-L., P.-K. Wong. 2004. Exploration vs. exploitation: An empirical test of the ambidexterity hypothesis. *Organ. Sci.* **15**(4) 481–494.
- Herbsleb, J. D., A. Mockus. 2003. An empirical study of speed and communication in globally distributed software development. *IEEE Trans. Software Engng.* **29**(6) 481–494.
- Herbsleb, J. D., D. J. Paulish, M. Bass. 2005. Global software development at Siemens: Experience from nine projects. *27th Internat. Conf. Software Engng.* Association for Computing Machinery, St. Louis.
- Highsmith, J. 1999. *Adaptive Software Development: A Collaborative Approach to Managing Complex Systems*. Dorset House Publishing Company, New York.
- Highsmith, J., A. Cockburn. 2001. Agile software development: The business of innovation. *IEEE Comput.* **9**(34) 120–122.
- Hinds, P. J., D. E. Bailey. 2000. Virtual teams: Anticipating the impact of virtuality on team process and performance. *Annual Meeting Academy of Management*. Toronto, Canada.
- Im, G., A. Rai. 2008. Knowledge sharing ambidexterity in long-term interorganizational relationships. *Management Sci.* **54**(7) 1281–1296.
- Jansen, J. J. P., F. A. J. V. D. Bosch, H. W. Volberda. 2006. Exploratory innovation, exploitative innovation, and ambidexterity: The impact of environmental and organizational antecedents. *Management Sci.* **52**(11) 1661–1674.
- Jansen, J. J. P., M. P. Tempelaar, F. A. J. V. D. Bosch, H. W. Volberda. 2009. Structural differentiation and ambidexterity: The mediating role of integration mechanisms. *Organ. Sci.* **20**(4) 797–811.
- Jarvenpaa, S. L., D. E. Leidner. 1998. Communication and trust in global virtual teams. *Organ. Sci.* **10**(6) 791–815.
- Komi-Sirvio, S., M. Tihinen. 2005. Lessons learned by participants of distributed software development. *Knowledge and Process Management* **12**(2) 108–122.
- Komi-Sirvio, S., P. Abrahamsson, T. Huomo. 2006. Guest editorial for the special section on distributed software development. *Inform. Software Tech.* **48**(9) 765–766.
- Kurland, N. B., T. D. Egan. 1999. Telecommuting: Justice and control in the virtual organization. *Organ. Sci.* **10**(4) 500–513.

- Layman, L., L. Williams, D. Damian, H. Bures. 2006. Essential communication practices for extreme programming in a global software development team. *Inform. Software Tech.* **48**(9) 781–794.
- Lee, G., W. Delone, J. A. Espinosa. 2006. Ambidextrous coping strategies in globally distributed software development projects. *Comm. ACM* **49**(10) 35–40.
- Lindvall, M., V. Basili, B. Boehm, P. Costa, K. Dangle, F. Shull, R. Tesoriero, L. Williams, M. Zelkowitz. 2002. Empirical findings in agile methods. D. Wells, L. Williams, eds. *Proc. XP/Agile Universe 2002: Second XP Universe and First Agile Universe Conf.*, Springer-Verlag GmbH, Chicago, 197–207.
- Lubatkin, M. H., Z. Simsek, L. Yan, J. F. Veiga. 2006. Ambidexterity and performance in small- to medium-sized firms: The pivotal role of top management team behavioral integration. *J. Management* **32**(5) 646–672.
- March, J. G. 1991. Exploration and exploitation in organizational learning. *Organ. Sci.* **2**(1) 71–87.
- Mason, J. 2002. *Qualitative Researching*, 2nd ed. Sage Publications, London.
- Matloff, N. 2005. Offshoring: What can go wrong? *IT Professional* **7**(4) 39–45.
- Meadows, C. J. 1996. Globalizing software development. *J. Global Inform. Management* **4**(1) 5–14.
- Miles, M. B., A. M. Huberman. 1994. *Qualitative Data Analysis*, 2nd ed. Sage Publications, Thousand Oaks, CA.
- Miller, A. 2008. Distributed agile development at Microsoft patterns and practices, <http://www.microsoft.com/downloads/details.aspx?displaylang=en&FamilyID=2143646d-bcc1-4d97-b9a1-96820191-7baa>.
- Mockus, A., J. Herbsleb. 2001. Challenges of global software development. *7th IEEE Internat. Software Metrics Sympos. (METRICS 2001)*. IEEE Computer Society, London, 182–184.
- Mom, T. J. M., F. A. J. V. D. Bosch, H. W. Volberda. 2009. Understanding variation in managers' ambidexterity: Investigating direct and interaction effects of formal structural and personal coordination mechanisms. *Organ. Sci.* **20**(4) 812–828.
- O'Reilly, III, C. A., M. L. Tushman. 2004. The ambidextrous organization. *Harvard Bus. Rev.* **82**(4) 74–81.
- Orlikowski, W. J. 1993. Case tools as organizational change: Investigating incremental and radical changes in systems development. *MIS Quart.* **17**(3) 309–340.
- Orlikowski, W. J., J. Baroudi. 1991. Studying IT in organizations: Research approaches and assumptions. *Inform. Systems Res.* **2**(1) 1–28.
- Oshri, I., J. Kotlarsky, L. P. Willcocks. 2007. Global software development: Exploring socialization in distributed strategic projects. *J. Strategic Inform. Systems* **16**(1) 25–49.
- Paré, G. 2004. Investigating information systems with positivist case study research. *Comm. AIS* **2004**(13) 233–264.
- Parnas, D. 2006. Agile methods and GSD: The wrong solution to an old but real problem. *Comm. ACM* **49**(10) 26–34.
- Patton, M. P. 1990. *Qualitative Evaluation and Research Methods*, 2nd ed. Sage, London.
- Pawlowski, S., D. Robey. 2004. Bridging user organizations: Knowledge brokering and the work of information technology professionals. *MIS Quart.* **28**(4) 645–672.
- Raisch, S., J. Birkinshaw. 2008. Organizational ambidexterity: Antecedents, outcomes, and moderators. *J. Management* **34**(3) 375–409.
- Ramesh, B., L. Cao, R. Baskerville. 2007. Agile requirements engineering practices and challenges: An empirical study. *Inform. Systems J.*
- Ramesh, B., L. Cao, K. Mohan, X. Peng. 2006. Can distributed software development be agile? *Comm. ACM* **49**(10) 41–46.
- Redmiles, D., A. Van Der Hoek, B. Al-Ani, T. Hildenbrand, S. Quirk, A. Sarma, R. Silva, C. De Souza, E. Trainer. 2007. A new paradigm to support globally distributed software development projects. *Wirtschaftsinformatik* **49** 28–38.
- Repenning, A., A. Ioannidou, M. Payton, W. Ye, J. Roschelle. 2001. Using components for rapid distributed software development. *IEEE Software* **18**(2) 38–45.
- Rothaermel, F. T., M. T. Alexandre. 2009. Ambidexterity in technology sourcing: The moderating role of absorptive capacity. *Organ. Sci.* **20**(4) 759–780.
- Simons, M. 2006. Global software development: A hard problem requiring a host of solutions. *Comm. ACM* **49**(10) 32–33.
- Slaughter, S. A., L. Levine, B. Ramesh, J. Pries-Heje, R. Baskerville. 2006. Aligning software processes with strategy. *MIS Quart.* **30**(4) 891–918.
- Stank, T. P., T. J. Goldsby, S. K. Vickery. 1999. Effect of service supplier performance on satisfaction and loyalty of store managers in the fast food industry. *J. Oper. Management* **17**(4) 429–447.
- Strauss, A., J. Corbin. 1990. *Basics of Qualitative Research: Grounded Theory Procedures and Techniques*. Sage Publication, London.
- Tiwana, A. 2008. Do bridging ties complement strong ties? An empirical examination of alliance ambidexterity. *Strategic Management J.* **29**(3) 251–272.
- Tushman, M. L., C. A. O'Reilly, III. 1996. Ambidextrous organizations: Managing evolutionary and revolutionary change. *California Management Rev.* **38**(4) 8–30.
- Vinekar, V., C. W. Slinkman, S. Nerur. 2006. Can agile and traditional systems development approaches coexist? An ambidextrous view. *Inform. Systems Management* **23**(3) 31–42.
- Wiesenfeld, B. M., S. Raghuram, R. Garud. 1998. Communication patterns as determinants of organizational identification in a virtual organization. *J. Comput.-Mediated Comm.* **3**(4). <http://onlinelibrary.wiley.com/doi/10.1111/j.1083-6101.1998.tb00081.x/abstract>.
- Wongthongtham, P., E. Chang, T. S. Dillon, I. Sommerville. 2006. Ontology-based multi-site software development methodology and tools. *J. Systems Architecture* **52**(11) 640–653.
- Yin, R. K. 2003. *Case Study Research: Design and Methods*, 3rd ed. Sage Publications, London.